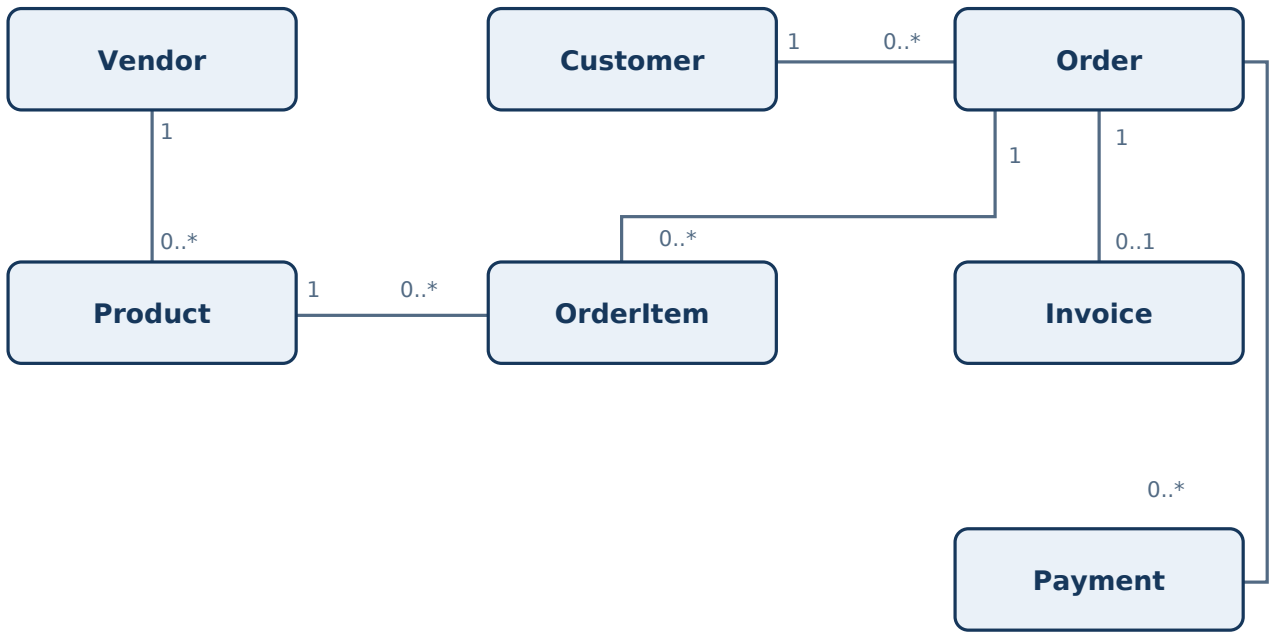
E-Commerce Database Design

Smaira Arora | Transactional Web Applications - 420-951-VA

Quiz 1: Database completion | October 4, 2026

1. Conceptual schema

An e-commerce store manages vendors, products, customers and purchases. An order consists of product lines, can have one invoice, and can record multiple payments. The model includes all seven required entities.



Parent -> child	Child count per parent	Parent count per child
Vendor -> Product	0..many	exactly 1
Customer -> Order	0..many	exactly 1
Order -> OrderItem	0..many	exactly 1
Product -> OrderItem	0..many	exactly 1
Order -> Invoice	0..1	exactly 1
Order -> Payment	0..many	exactly 1

0..* means zero or more; 0..1 means optional and unique. OrderItem resolves the many-to-many relationship between Order and Product. Completed purchases require at least one order item; this rule belongs to the purchase service.

2. Logical schema and data dictionary

PK = primary key; FK = foreign key. Identity keys are generated by SQL Server. All fields are NOT NULL except those marked optional.

Vendor

Column	Physical type	Logical rules
vendor_id	BIGINT IDENTITY(1,1)	PK
name	NVARCHAR(100)	required; nonblank
email	NVARCHAR(254)	required; unique
phone	NVARCHAR(30)	optional
is_active	BIT	required; default 1

Customer

Column	Physical type	Logical rules
customer_id	BIGINT IDENTITY(1,1)	PK
first_name	NVARCHAR(60)	required; nonblank
last_name	NVARCHAR(60)	required; nonblank
email	NVARCHAR(254)	required; unique
phone	NVARCHAR(30)	optional
is_active	BIT	required; default 1

Product

Column	Physical type	Logical rules
product_id	BIGINT IDENTITY(1,1)	PK
vendor_id	BIGINT	required; FK Vendor
sku	NVARCHAR(50)	required; unique
name	NVARCHAR(120)	required; nonblank
description	NVARCHAR(1000)	optional
price	DECIMAL(19,2)	required; > 0
quantity	INT	required; >= 0; default 0
status	VARCHAR(20)	required; trigger maintained
is_active	BIT	required; default 1

2. Logical schema and data dictionary

PK = primary key; FK = foreign key. Identity keys are generated by SQL Server. All fields are NOT NULL except those marked optional.

Order

Column	Physical type	Logical rules
order_id	BIGINT IDENTITY(1,1)	PK
customer_id	BIGINT	required; FK Customer
order_date	DATETIME2(0)	required; UTC default
status	VARCHAR(20)	required; default PENDING
total_amount	DECIMAL(19,2)	required; >= 0
currency	CHAR(3)	required; CAD only

OrderItem

Column	Physical type	Logical rules
order_item_id	BIGINT IDENTITY(1,1)	PK
order_id	BIGINT	required; FK Order
product_id	BIGINT	required; FK Product
quantity	INT	required; > 0
unit_price	DECIMAL(19,2)	required; > 0; price snapshot
line_total	DECIMAL(19,2)	computed; quantity * unit_price

Additional uniqueness: OrderItem(order_id, product_id) prevents duplicate product lines. Invoice.order_id guarantees at most one invoice per order. Every table has an identity primary key.

Historical price: unit_price stores the checkout price even if Product.price changes. line_total is computed as DECIMAL(19,2). Order.total_amount must equal the sum of its item totals; the service maintains this cross-table rule in one transaction.

2. Logical schema and data dictionary

PK = primary key; FK = foreign key. Identity keys are generated by SQL Server. All fields are NOT NULL except those marked optional.

Invoice

Column	Physical type	Logical rules
invoice_id	BIGINT IDENTITY(1,1)	PK
order_id	BIGINT	required; unique; FK Order
invoice_number	NVARCHAR(40)	required; unique
issued_at	DATETIME2(0)	required; UTC default
total_amount	DECIMAL(19,2)	required; >= 0
status	VARCHAR(20)	required; default ISSUED

Payment

Column	Physical type	Logical rules
payment_id	BIGINT IDENTITY(1,1)	PK
order_id	BIGINT	required; FK Order
method	VARCHAR(20)	required; CARD or BANK
amount	DECIMAL(19,2)	required; > 0
status	VARCHAR(20)	required; default SUCCESS
reference	NVARCHAR(80)	required; unique
paid_at	DATETIME2(0)	required; UTC default

Allowed states: Product: IN_STOCK / OUT_OF_STOCK. Order: PENDING / PAID / CANCELLED. Invoice: ISSUED / PAID / VOID. Payment: SUCCESS / FAILED / REFUNDED. Payment methods: CARD / BANK. Currency: CAD.

Normalization: Vendor and customer details are stored once. OrderItem links orders to products, so there are no repeating product columns. Invoice and payment records have separate identities. Price snapshots, cached document totals and product status are deliberate historical/derived values with explicit maintenance rules.

3. Physical implementation

The database uses SQL Server dbo tables, BIGINT identity keys, Unicode NVARCHAR text, DECIMAL(19,2) money, INT inventory, BIT active flags and DATETIME2(0) UTC timestamps. The reserved word Order is bracketed in T-SQL.

Script	Purpose
create_database.sql	Create ECommerceDB if absent.
create_tables.sql	Create seven tables, named constraints and FK indexes.
create_triggers.sql	Synchronize product status for all inserted/updated rows.
insert_sample_data.sql	Load sample data and commit a complete paid purchase.
verify_database.sql	Assert schema, test multi-row trigger, and force rollback.

Integrity and deletion policy

Primary and foreign keys prevent orphaned records. Unique constraints protect emails, SKU, invoice numbers, payment references, invoice order IDs and order/product pairs. CHECK constraints restrict amounts, quantities, states and nonblank identifiers. Foreign keys use NO ACTION; deactivate referenced vendors, customers and products rather than deleting historical data.

Multi-row inventory trigger

The AFTER INSERT, UPDATE trigger joins Product to every row in inserted. Quantity 0 produces OUT_OF_STOCK; positive quantity produces IN_STOCK. Restocking also changes status. Only mismatched rows are updated, so recursive execution terminates. Status synchronization is in the same database transaction as the inventory write.

Transaction, rollback and concurrency

The sample-data script commits order, items, inventory, invoice and payment together. The verification script deliberately throws an insufficient-payment error after all related writes and asserts that counts, stock and status return to their previous values. Identity gaps after rollback are normal. Run this test without concurrent edits.

Inventory is read with UPDLOCK and HOLDLOCK inside the test transaction. For the later Spring service, acquire pessimistic write locks inside @Transactional, lock product IDs in ascending order, and validate inventory before decrementing. Every purchase path must follow this discipline to avoid overselling.

Scope, assumptions and verification

This submission covers the database milestone. CAD is the only currency; taxes, discounts and shipping are outside this design. Email-format checks, active-account rules, total consistency and authorization require application validation. Later milestones add JPA, Spring Security, API and React.

SQL Server execution was unavailable in the authoring environment. Run the supplied verification script in SSMS; its expected PASS messages are documented in README.md. No execution results or screenshots are claimed.

References: instructor project description; Microsoft Learn, Create DML Triggers to Handle Multiple Rows of Data; Microsoft Learn, Table Hints (Transact-SQL). Links are included in README.md.